PQC DAILY STUDY SERIES

PQC for Resource-Constrained Environments: IoT and Embedded Systems

Day 35 | Category: PQC Adoption & Industry Status Date: 2026-10-19 | DAILY MONITORING

PQC for Resource-Constrained Environments: IoT and Embedded Systems

The Challenge of "Big" Crypto on "Small" Devices

As the world transitions to Post-Quantum Cryptography (PQC), one of the most significant hurdles is its implementation on the billions of Internet of Things (IoT) and embedded devices. These devices, from smart home sensors to industrial controllers, are typically designed with strict constraints on processing power (CPU), memory (RAM), and power consumption. However, many of the standardized PQC algorithms, particularly ML-KEM and ML-DSA, were designed primarily for performance on servers and desktop-class machines, leading to new challenges in the embedded world.

Why is PQC on IoT a Unique Problem?

Unlike a web server that can dedicate significant resources to a TLS handshake, an IoT device must perform cryptographic operations while also running its primary functions, often on a battery-powered budget. The key challenges are:

- **Larger Key Sizes:** PQC algorithms generally have much larger public keys and ciphertexts/signatures compared to their classical counterparts (e.g., ECC). This increases RAM requirements for storage and transmission bandwidth, a critical factor for low-power, wide-area networks (LPWANs).
- **Higher Computational Cost:** The mathematical operations in lattice-based cryptography (like ML-KEM and ML-DSA) are more complex than ECC, demanding more CPU cycles. This can lead to increased latency and higher energy drain.
- **Firmware Size (Flash Memory):** The code size of PQC libraries can be substantial. Fitting these new libraries alongside the operating system and application logic into the limited flash memory of a microcontroller is a non-trivial task.

Strategies and Optimizations for Embedded PQC

Researchers and engineers are actively developing solutions to make PQC viable for constrained devices. The primary focus is on software optimization and, increasingly, hardware acceleration.

PQC Implementation Approaches for IoT

Approach	Description	Pros	Cons
Optimized Software Libraries	Developing lightweight versions of PQC libraries (like liboqs or mbedTLS) specifically tailored for embedded architectures (e.g., ARM Cortex-M). This involves using less RAM, optimizing instruction sets (like ARMv8-M's DSP extensions), and carefully managing memory allocation.	- No hardware changes needed - Faster to deploy	- Performance may still be insufficient for real-time applications - High energy consumption
Hardware Acceleration	Designing dedicated co-processors or custom instructions within the CPU to handle the most intensive PQC operations, such as Number Theoretic Transform (NTT) for lattice cryptography. This offloads the main processor.	- Massive performance gains (10x-100x) - Significantly lower energy consumption	- Requires new silicon (ASICs/FPGAs) - Increases device cost and design time
Algorithm Selection	For some applications with extreme constraints, selecting PQC algorithms that were not standardized by NIST but are designed for IoT may be an option, though this carries interoperability and security-vetting risks. The focus remains on optimizing the NIST standards.	- Can be tailored to specific constraints	- Lack of standardization - Reduced interoperability and peer review

Real-World Applicability: Securing the Edge

The successful deployment of PQC in IoT is critical for long-term security. Consider a smart electrical grid or a network of medical devices. These systems may be in the field for 10-20 years. If they are deployed today with only classical cryptography, they are vulnerable to "Harvest Now, Decrypt Later" attacks. By integrating PQC, we ensure their communications—from sensor readings to control commands and firmware updates (signed with ML-DSA)—remain secure for their entire operational lifespan, even in a quantum future.

Daily Quiz

1. What are the three main resource constraints in IoT devices that make PQC implementation challenging?
2. What is the primary trade-off between using an optimized software library versus dedicated hardware acceleration for PQC on an embedded device?
3. Why is the "Harvest Now, Decrypt Later" (HNDL) threat particularly relevant for IoT systems?
